

ASS 19.4

2303A52346

JavaScript to Python (Array Processing Logic)

Scenario

A frontend application written in JavaScript processes user scores. The

backend team wants the same logic implemented in Python for analytics

processing.

Task

Translate a JavaScript function that:

- Takes an array of numbers
- Filters values greater than 50
- Returns the average of filtered values

Instructions

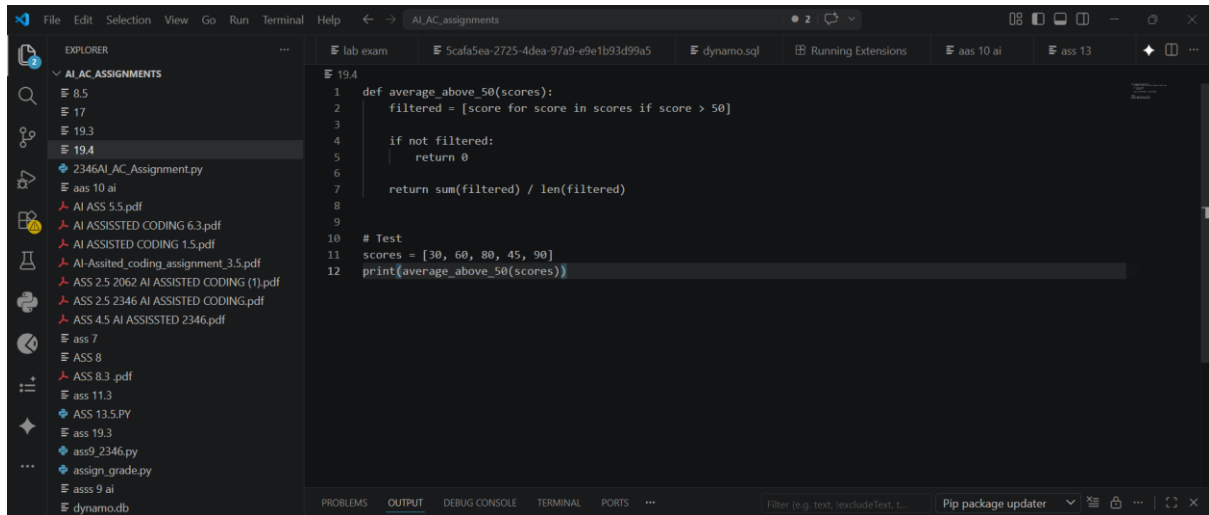
- Prompt AI to convert JavaScript array methods (filter, reduce) into

equivalent Python logic.

- Ensure list comprehension or Pythonic style is used.
- Compare outputs of both implementations with sample input.
- Document differences in handling arrays between JS and Python.

Expected Output

- Equivalent Python function.
- Same output for identical inputs.
- Clean and Pythonic implementation.



```
1 def average_above_50(scores):
2     filtered = [score for score in scores if score > 50]
3
4     if not filtered:
5         return 0
6
7     return sum(filtered) / len(filtered)
8
9
10 # Test
11 scores = [30, 60, 80, 45, 90]
12 print(average_above_50(scores))
```

Python OOP to C++ Class Translation

Scenario

A data modeling module is written in Python using OOP. The system is being

migrated to a high-performance C++ environment.

Task

Translate a Python class that:

- Contains private attributes
- Uses getter/setter methods
- Includes a method that calculates a derived value

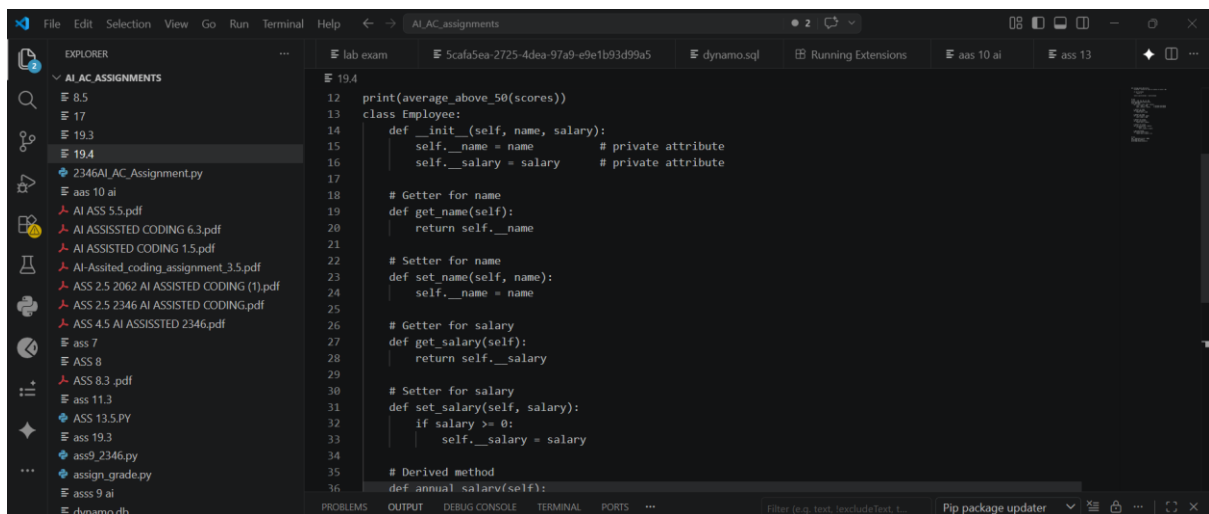
Instructions

- Convert the Python class into a C++ class.

- Ensure:
 - o Proper access modifiers
 - o Constructor handling
 - o Correct data types
- Compile and test the C++ version.
- Compare how encapsulation differs between Python and C++.

Expected Output

- Fully working C++ class equivalent to Python version.
- Proper use of access specifiers (private, public)



```

12 print(average_above_50(scores))
13 class Employee:
14     def __init__(self, name, salary):
15         self.__name = name        # private attribute
16         self.__salary = salary    # private attribute
17
18     # Getter for name
19     def get_name(self):
20         return self.__name
21
22     # Setter for name
23     def set_name(self, name):
24         self.__name = name
25
26     # Getter for salary
27     def get_salary(self):
28         return self.__salary
29
30     # Setter for salary
31     def set_salary(self, salary):
32         if salary >= 0:
33             self.__salary = salary
34
35     # Derived method
36     def annual_salary(self):
  
```

A backend developer wrote an API request using Python requests. The

frontend team needs the same functionality in JavaScript (Fetch API).

Task

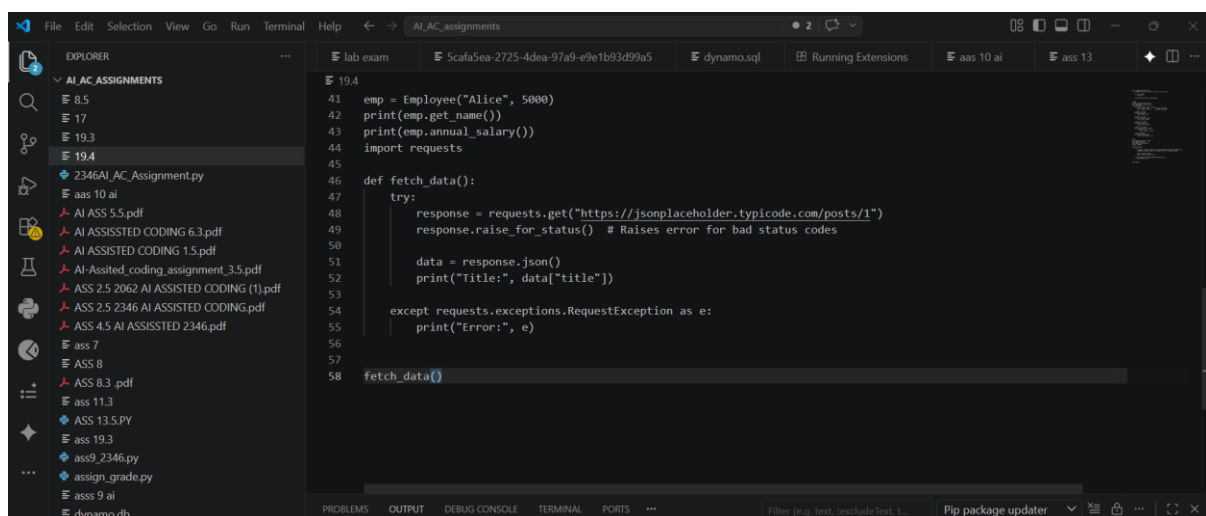
Translate Python API request code into JavaScript.

Instructions

- Convert:
 - o GET request logic
 - o JSON parsing
 - o Error handling (try-except → try-catch)
- Ensure proper async/await usage in JavaScript.
- Compare differences in error handling mechanisms.

Expected Output

- Equivalent JavaScript code using Fetch API.
- Proper async handling.
- Similar output structure.



The screenshot shows a VS Code editor window with the file explorer on the left and a code editor in the center. The file explorer shows a folder named 'AI_AC_ASSIGNMENTS' with several files, including '2346AI_AC_Assignment.py'. The code editor shows a Python script with the following code:

```
19.4
41 emp = Employee("Alice", 5000)
42 print(emp.get_name())
43 print(emp.annual_salary())
44 import requests
45
46 def fetch_data():
47     try:
48         response = requests.get("https://jsonplaceholder.typicode.com/posts/1")
49         response.raise_for_status() # Raises error for bad status codes
50
51         data = response.json()
52         print("Title:", data["title"])
53
54     except requests.exceptions.RequestException as e:
55         print("Error:", e)
56
57
58 fetch_data()
```

